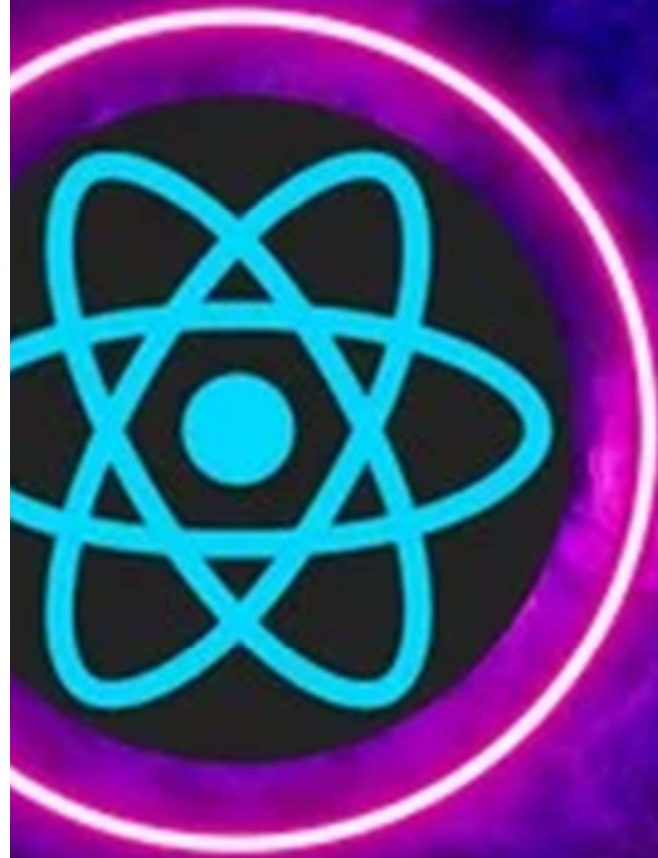


JavaScript-15

Avancerad React



30 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg



För en kanin

Avancerad React JS

Nu när du vet hur du installerar React och hur en enskild komponent fungerar, ska vi titta på hur man bygger appar på riktigt – genom att skicka data mellan komponenter och hämta information från ett externt REST API.

Props – Skicka data mellan Lego-bitar

I React vill vi ofta återanvända komponenter. Tänk dig att du vill visa tre olika produktkort på en sida. Istället för att skapa tre olika filer, skapar vi en komponent som heter ProduktKort, och sedan skickar vi med olika information till den.

Informationen vi skickar med kallas för Props (properties). Det fungerar precis som när du skickar med argument in i en vanlig JavaScript-funktion.

Här skapar vi först en liten underkomponent som tar emot props:

```
// En fristående komponent som tar emot ett objekt (props)
function ProduktKort(props) {
  return (
    <div className="kort">
      <h3>{props.titel}</h3>
      <p>Pris: {props.pris} kr</p>
    </div>
  );
}
```

Bild 1 En fristående komponent i React.

Inuti vår huvudfil (App.jsx) kan vi nu använda den här komponenten flera gånger och skicka med olika props till den.

```
function App() {
  return (
    <div className="app-container">
      <h1>Vår Butik</h1>

      {/* Vi skickar med data som attribut i JSX */}
      <ProduktKort titel="Blå Jeans" pris={799} />
      <ProduktKort titel="Röd T-shirt" pris={299} />
    </div>
  );
}
```

Bild 2 Props gör att samma komponent kan visa helt olika innehåll..

Hämta data från ett API (Fetch i React)

I dina tidigare kapitel lärde du dig hur man hämtar data med `fetch()`. I React kan vi inte bara kasta ut ett `fetch`-anrop hur som helst inuti en komponent, eftersom komponenten ritas om varje gång ett state ändras (vilket skulle starta en oändlig loop av API-anrop!).

För att styra när kod ska köras använder vi en inbyggd React-funktion som heter `useEffect`. Vi talar om för React: "Kör det här `fetch`-anropet en enda gång, precis när sidan laddas för första gången". Här hämtar vi en lista med användare från ett gratis REST-API (JSONPlaceholder).

```
import { useState, useEffect } from 'react';

function AnvändarLista() {
  const [användare, setAnvändare] = useState([]); // Startar med en tom array

  // useEffect tar två argument: en funktion och en "dependency-array" []
  useEffect(() => {

    // Vi gör vårt fetch-anrop inuti useEffect
    fetch('https://typicode.com')
      .then(response => response.json())
      .then(data => {
        setAnvändare(data); // Vi sparar datan i vårt state!
      });
  });
}
```

```
    }, []); // Den tomma klammern [] i slutet betyder: "Kör bara EN gång vid sidstart"

    return (
      <div>
        <h2>Våra Kunder</h2>
        <ul>
          { /* Vi använder .map() från kapitel 06 för att loopa ut datan! */ }
          { användare.map(user => (
            <li key={user.id}>{user.name} ({user.email})</li>
          )) }
        </ul>
      </div>
    );
  }
}
```

Bild 3 Kombinerat useEffect för att hämta data och useState för att spara den.

Hur State fungerar under huven (Det dolda flödet)

Om du skapar en helt vanlig variabel inuti en React-komponent, till exempel `let klick = 0;`, och ökar den när någon klickar på en knapp, kommer ingenting att hända på skärmen. Variabeln ökar i datorns minne, men webbläsaren visar fortfarande en nolla.

Varför? Jo, för att React inte ligger och spionerar på dina vanliga variabler. Webbläsaren ritas bara om skärmen när React kör om (åter-renderar) hela din komponent-funktion. När vi använder `useState` skapar vi en speciell variabel som är kopplad direkt till Reacts motor. Varje gång du anropar din "set-funktion" (t.ex. `setAntal`), så gör React två saker i bakgrunden:

1. Den uppdaterar värdet på variabeln.
2. Den kastar omedelbart allt den har i händerna och kör om hela din funktion så att den nya siffran ritas ut på skärmen.

STRECK 1: Användaren klickar på en knapp.



STRECK 2: Kodan anropar: `setAntal(antal + 1)`



STRECK 3: React uppdaterar värdet i sitt dolda minne.



STRECK 4: ⚡ RE-RENDER! React kör om hela din komponent och ritar blixtsnabbt ut den nya siffran på skärmen.

Bild 4 State är inte bara en variabel – det är en avtryckare som tvingar React att uppdaterar gränssnittet i realtid.

Hantera asynkron data utan att krascha appen

När vi hämtar data från ett externt API (till exempel med Axios som vi lärde oss i kapitel 04b) tar det alltid en liten stund innan svaret kommer tillbaka över internet. Detta skapar ett jättevanligt problem i React. Om din JavaScript-kod försöker loopa ut produkterna med `.map()` innan servern har svarat, kommer din app att tvärkrascha direkt med det klassiska felmeddelandet: `TypeError: Cannot read properties of undefined (reading 'map')`.

För att skriva robust och professionell kod måste vi hantera tre helt olika tillstånd i vår komponent: Laddar, Data, och Fel.

Här kombinerar vi `useEffect` och `useState` för att skapa ett skottsäkert flöde som visar en laddningstext medan internetlinan jobbar, och ett snyggt felmeddelande om servern skulle ligga nere.

```
import { useState, useEffect } from 'react';
import axios from 'axios';

function ProduktUtbud() {
  const [produkter, setProdukter] = useState([]); // Startar som en tom lista
  const [laddar, setLaddar] = useState(true);      // Startar som true eftersom
  vi laddar vid start
  const [fel, setFel] = useState(null);           // Startar som null (inget fel
  än)

  useEffect(() => {
    async function hämtaData() {
```

```

    try {
      // Vi gör anropet med Axios (som vi lärde oss i kapitel 04b!)
      const response = await axios.get('https://fakestoreapi.com');
      setProdukter(response.data); // Spara datan i vårt state
    } catch (error) {
      // Om Axios möter t.ex. ett 404-fel hamnar vi här direkt
      setFel("Kunde inte hämta produkterna. Försök igen senare.");
    } finally {
      setLaddar(false); // Stäng av laddningen oavsett om det gick bra eller
dåligt
    }
  }

  hämtaData();
}, []); // Den tomma klammern [] ser till att anropet bara körs EN gång vid
sidstart

// --- HÄR STYR VI VAD SOM VISAS PÅ SKÄRMEN BASERAT PÅ VÅRT STATE ---
if (laddar) return <h3>⌚ Laddar lagersaldo, vänligen vänta...</h3>;
if (fel) return <h3 style={{ color: 'red' }}>🚫 Fel: {fel}</h3>;

return (
  <div className="produkter-box">
    <h2>Vårt Aktuella Lager ({produkter.length} st varor)</h2>
    <ul>
      {produkter.map(produkt => (
        <li key={produkt.id}>
          <b>{produkt.title}</b> - {produkt.price} kr
        </li>
      ))}
    </ul>
  </div>
);
}

export default ProduktUtbud;

```

Bild 5 Genom att lägga if-satser som avbryter och returnerar HTML tidigt i funktionen, skyddar vi vår .map()-loop från att köras innan datan faktiskt har landat i vårt state.